

ORF307_HW7_soln

April 11, 2022

1 ORF307 Homework 7 Solution

2 Q1

- (a) We can see both x^* and y^* are feasible in primal and dual LPs, and $c^T x^* = -b^T y^*$. By optimality condition, we know x^* and y^* are optimal when $\epsilon = 0$.
- (b) we can rewrite the primal and dual LPs as following,

$$\begin{array}{ll} \text{maximize} & -(c + \epsilon d)^T x \\ \text{subject to} & Ax \leq b \end{array} \qquad \begin{array}{ll} \text{minimize} & b^T y \\ \text{subject to} & A^T y = -c - \epsilon d \\ & y \geq 0 \end{array}$$

Now we we can define new primal and dual LPs with primal being the original dual, while dual being the original primal.

$$\begin{array}{ll} \text{minimize} & b^T y \\ \text{subject to} & A^T y = -c - \epsilon d \\ & y \geq 0 \end{array} \qquad \begin{array}{ll} \text{maximize} & -(c + \epsilon d)^T x \\ \text{subject to} & Ax \leq b \end{array}$$

Now we define the optimal value of our new primal and dual LPs as $\tilde{p}^*$ and $\tilde{q}^*$. Then around $\epsilon = 0$ we have $p^*(\epsilon) = q^*(\epsilon) = \tilde{p}^*(\epsilon) = \tilde{p}^*(0) + (\epsilon d)^T x^* = -4 + 8\epsilon$

- (c) Based on **modified optimal solution** in lecture 15, page 28, we have

$$y^*(\epsilon) = y^* - \epsilon(A^T)^{-1}d = \begin{bmatrix} 0.5 + 0.5\epsilon \\ 0.5 - 2.5\epsilon \end{bmatrix} x^*(\epsilon) = x^*$$

3 Q2

- (a)

$$\begin{array}{ll} \text{minimize} & -60x_1 - 30x_2 - 20x_3 \\ \text{subject to} & 8x_1 + 6x_2 + x_3 \leq 48 + t \\ & 4x_1 + 2x_2 + 1.5x_3 \leq 20 + t \\ & 2x_1 + 1.5x_2 + 0.5x_3 \leq 8 + 5t \\ & x_1, x_2, x_3 \geq 0 \end{array}$$

```

[2]: import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt

x = cp.Variable(3)
t = cp.Parameter()
A = np.array([[8, 6, 1], [4, 2, 1.5], [2, 1.5, 0.5]])
b = np.array([48, 20, 8])
d = np.array([1, 1, 5])
c = np.array([-60, -30, -20])
prob = cp.Problem(cp.Minimize(c.T@x), [A@x <= b + t*d.T, x >= 0])

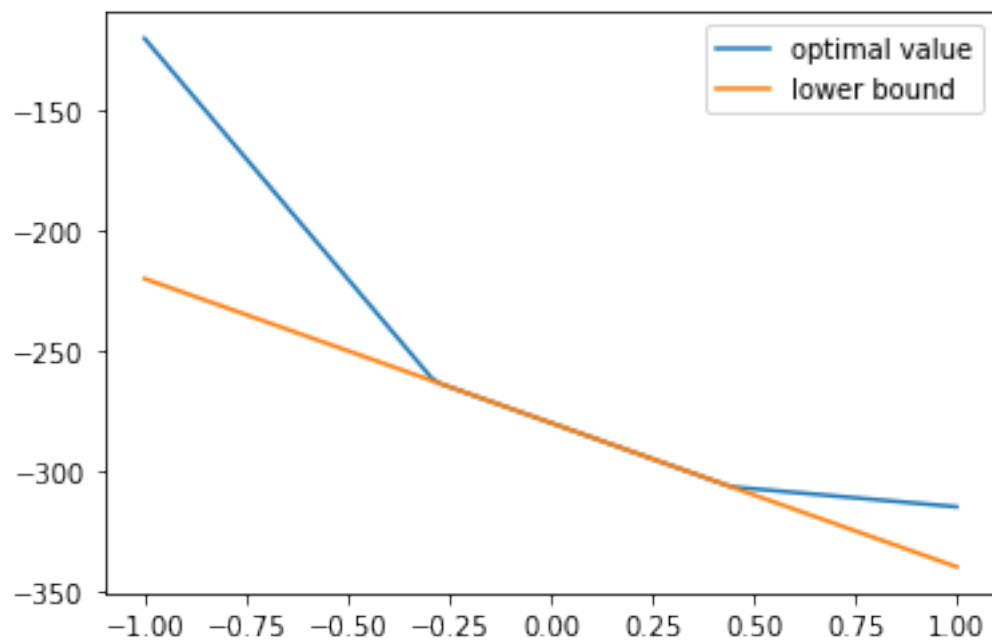
t_values = np.linspace(-1, 1, 100)
x_vector = []
objective_vector = []
for t_val in t_values:
    t.value = t_val
    prob.solve()
    x_vector.append(x.value)
    objective_vector.append(prob.objective.value)

t.value = 0
prob.solve()
p0 = prob.objective.value

y = cp.Variable(3)
dual_prob = cp.Problem(cp.Maximize(-b.T@y), [A.T@y >= -c, y >= 0])
dual_prob.solve()
lower_bound = p0 - t_values * (d.T@y.value)
plt.plot(t_values, objective_vector, label='optimal value')
plt.plot(t_values, lower_bound, label='lower bound')
plt.legend()

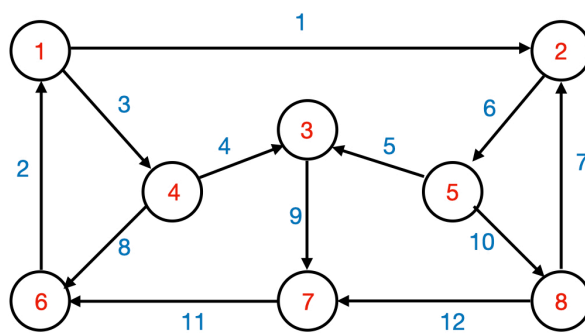
```

[2]: <matplotlib.legend.Legend at 0x12af48fd0>



4 Q3

- (a) We have 8 nodes and 12 arcs, and their indices are labeled as below. Then we define matrix A and LP as following



```
[3]: import cvxpy as cp
x = cp.Variable(12)
A = np.array([[1, -1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
              [-1, 0, 0, 0, 0, 1, -1, 0, 0, 0, 0, 0],
              [0, 0, 0, -1, -1, 0, 0, 0, 1, 0, 0, 0],
              [0, 0, -1, 1, 0, 0, 0, 1, 0, 0, 0, 0],
              [0, 0, 0, 0, 1, -1, 0, 0, 0, 1, 0, 0],
              [0, 1, 0, 0, 0, 0, 0, -1, 0, 0, -1, 0],
              [0, 0, 0, 0, 0, 0, 0, 0, -1, 0, 1, -1],
              [0, 0, 0, 0, 0, 0, 1, 0, 0, -1, 0, 1]])
b = np.array([2, 0, 2, -1, 0, -2, -6, 5])
c = np.array([1, 0, 0, 2, 3, 1, 1, 3, 2, 0, 1, 4])
prob = cp.Problem(cp.Minimize(c.T*x), [A*x == b, x >= 0])
prob.solve()
print('minimum cost:', prob.objective.value)
print(np.round(x.value))
```

```
minimum cost: 27.999999999999595
[-0. -0.  2.  0.  0.  0.  0.  1.  2.  0.  1.  5.]
```

```
[ ]:
```